Exercise for MA-INF 2201/MA-MOROB-M04 Computer Vision WS26/27
Submission on 25.10.2026

Overview

In this assignment, you are required to write code for the programming tasks in Questions 1, 3 and 4. We recommend using Python 3.12, NumPy 2.3.3, and OpenCV 4.11.0.86. You may only use NumPy, OpenCV, matplotlib, and scikit-image (scikit-image for computing the PSNR in Question 1, Parts 1.2 and 1.3 only). No other external libraries are permitted. Submit your Python files (.py or Jupyter notebook) with clear comments indicating which task each section addresses. Complete the provided templates for Questions 1, 3 and 4 (`q1_template.py`, `q3_template.py`, `q4_template.py`). Also submit a PDF containing your answer to Question 2 and all discussion sections.

1 Denoising and Optimization (8 points)

Read the images `bonn.jpg` and `bonn_noisy.jpg` and convert them to grayscale images. The first image is the original image and the second one has been corrupted with mixed noise (Gaussian and Salt-and-Pepper noise). Your task is to denoise the noisy image according to the following instructions.

1.1 Denoising Filters

Denoise the noisy image with each of the following filters:

- **Gaussian filter** (from scratch: convolution with a Gaussian kernel),
- **Median filter** (from scratch: median of each neighborhood),
- **Bilateral filter** (from scratch: weighting with a domain and a range kernel).

For each filter, provide two solutions: one using the built-in `cv2` function and one implemented by you from scratch. Display the resulting image for each applied filter.

Background: Peak Signal-to-Noise Ratio (PSNR)
PSNR is a metric used to measure the quality of reconstructed or denoised images compared to the original image. It quantifies the ratio between the maximum possible signal power and the noise power. Higher PSNR values indicate better image quality (less noise/distortion). It is defined as

$$\text{PSNR} = 10 \cdot \log_{10}\left(\frac{\text{MAX}^2}{\text{MSE}}\right), \qquad \text{MSE} = \frac{1}{mn}\sum_{i=1}^{m}\sum_{j=1}^{n}\big[I(i,j) - K(i,j)\big]^2,$$

where MAX is the maximum possible pixel value (255 for 8-bit grayscale images), I is the original image, K is the denoised (or noisy) image, and $m \times n$ is the image size.
The PSNR can be computed with skimage:

```
from skimage.metrics import peak_signal_noise_ratio as psnr
psnr_value = psnr(original_image, denoised_image, data_range=255)
```

Set `data_range=255` for 8-bit images (pixel values 0–255).

1.2 Performance Comparison

Compare which filter performs the best at denoising using the PSNR metric. Calculate PSNR values using the provided skimage library code for all three filters.

1.3 Parameter Optimization

For all three filters, write a function that iterates over relevant parameters (e.g., kernel size for Median/Gaussian; σ_d, σ_r for Bilateral) to find the optimal parameter values that maximize PSNR. After finding the best parameters through your optimization function, report them clearly in your code as variables or constants, and use these values to generate your final denoised images. During grading, the optimization code may be commented out to verify that the reported parameters produce the claimed PSNR values.

1.4 Discussion

The input image contains mixed Gaussian and Salt-and-Pepper noise. Based on your PSNR results and visual inspection, discuss which filter handled the noise most effectively. Explain why certain filters are theoretically better suited for specific noise types, and whether your experimental results aligned with these expectations (*please limit your response to 3–5 sentences*).

2 Convolution of Gaussians (4 points)

Let the 1D Gaussian with standard deviation σ be

$$g_\sigma(x) = \frac{1}{\sqrt{2\pi}\,\sigma} \exp\left(-\frac{x^2}{2\sigma^2}\right).$$

2.1 Convolution of Two 1D Gaussians

Prove that the convolution of two Gaussians is again a Gaussian:

$$(g_{\sigma_1} * g_{\sigma_2})(x) = g_\sigma(x), \qquad \sigma = \sqrt{\sigma_1^2 + \sigma_2^2}.$$

Conclude that convolving twice with a Gaussian of standard deviation σ is the same as convolving once with a Gaussian of standard deviation $\sigma\sqrt{2}$.

Hint: Complete the square in the exponent and use $\int_{-\infty}^{\infty} e^{-a(u-b)^2}\,du = \sqrt{\pi/a}$ for $a > 0$.

2.2 Repeated Small Filter vs. One Large Filter

You want to smooth an image with a Gaussian of $\sigma = 4$, but only a Gaussian filter with $\sigma = 1$ is available. How many times do you have to apply it? Using the rule of thumb from the lecture (filter half-width $\approx 3\sigma$), compare the number of multiplications per pixel of the repeated small filter with a single large filter, both for a full 2D kernel and for a separable implementation. Which approach is cheaper?

3 Integral Images (3 points)

Read the image `bonn.jpg` and convert it into a grayscale image.

3.1 Integral Image

Implement a function to calculate the integral image.

3.2 Mean Using the Integral Image

Implement a function to compute the mean gray value of a specific region using the integral image. The region is defined by its top left corner at $(10, 10)$ and bottom right corner at $(60, 80)$.

3.3 Mean by Direct Summation

Implement a function to calculate the mean gray value of the same region by dividing the sum of all pixel intensities in that region by the total number of pixels in the region.

3.4 Computational Complexity

Analyze the computational complexity of both methods (Parts 3.2 and 3.3) by providing their theoretical time complexity (Big-O notation) and measuring their execution times in your implementation.

4 Histogram Equalization (5 points)

Read the image `bonn_lowcontrast.png`, a grayscale image with low contrast.

4.1 Histogram and CDF

Implement a function that computes the normalized histogram $p_f(y)$ and the cumulative distribution function $F_f(y)$ of the image without using `cv2.calcHist`, `np.histogram` or `np.bincount`. Plot both.

4.2 Histogram Equalization

Implement histogram equalization from scratch using the mapping $h(y) = \text{round}\big(255 \cdot F_f(y)\big)$. Apply the mapping as a lookup table and display the original and the equalized image.

4.3 Comparison with OpenCV

Apply `cv2.equalizeHist` to the same image and print the maximum absolute pixel difference to your result. If the results are not identical, explain where the difference comes from and adapt your implementation so that it reproduces the OpenCV result.

4.4 Histogram after Equalization

Plot the histogram of the equalized image. In the lecture we have shown that, for continuous intensities, the equalized histogram is flat (for 256 discrete levels this would be $p_h(z) = \frac{1}{256}$). Explain why the histogram of your result is nevertheless not flat.